

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA5112 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO5: Develop the network security strategies based on the study of viruses, worms, firewall architectures, and IDS/IPS functionalities.CO (BL5 , BL6)

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A

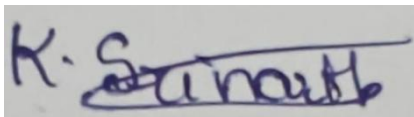
PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Code of Conduct:

I ____K.Srinath(192524220)_____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A handwritten signature in blue ink, reading "K. Srinath", is shown on a light gray background.

Annexure A

PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

1. STATEFUL FIREWALL SYSTEM

Design pseudocode for a stateful firewall system that tracks active connections and filters packets based on session state. Evaluate how state tracking improves security compared to stateless filtering.

PSEUDOCODE

ALGORITHM StatefulFirewall

BEGIN

Initialize StateTable

Initialize FirewallRules

WHILE firewall is active DO

Packet = ReceivePacket()

IF Packet is NULL THEN

CONTINUE

END IF

ConnectionKey = CreateConnectionKey(Packet)

Connection = Search(StateTable, ConnectionKey)

IF Connection exists THEN

IF Connection.State = ESTABLISHED OR

Connection.State = RELATED THEN

IF PacketMatchesConnection(Packet, Connection) AND

RuleAllows(Packet, FirewallRules) THEN

ALLOW Packet

UpdateConnection(StateTable, Connection, Packet)

ELSE

DROP Packet

END IF

ELSE

DROP Packet

END IF

ELSE

```
IF IsNewConnection(Packet) AND
RuleAllows(Packet, FirewallRules) THEN
AddConnection(StateTable, Packet, NEW)
ALLOW Packet
UpdateConnection(StateTable, ConnectionKey, Packet)
ELSE
DROP Packet
END IF
END IF
RemoveExpiredConnections(StateTable)
END WHILE
END
```

EVALUATION

State tracking improves security because the firewall does not examine each packet independently. It remembers active sessions and checks whether incoming packets belong to a valid connection. This allows it to identify unexpected, invalid, or unrelated packets and block them.

A stateless firewall evaluates each packet separately using predefined rules. It does not remember previous packets or the state of a connection. A stateful firewall therefore provides better session awareness and can make more context-based filtering decisions. The main trade-off is that maintaining a state table requires additional memory and processing.

2. SIGNATURE-BASED MALWARE DETECTION SYSTEM

Design pseudocode for a signature-based malware detection system that scans files and compares them against a known signature database. Evaluate its effectiveness in detecting known threats and its limitations against new attacks.

PSEUDOCODE

ALGORITHM SignatureBasedMalwareDetection

BEGIN

Load SignatureDatabase

FOR EACH File in FilesToScan DO

IF File is accessible THEN

FileSignature = GenerateSignature(File)

MatchFound = FALSE

FOR EACH Signature in SignatureDatabase DO

IF FileSignature = Signature THEN

MatchFound = TRUE

BREAK

END IF

END FOR

IF MatchFound = TRUE THEN

Mark File as MALWARE

Quarantine(File)

LogDetection(File)

ELSE

Mark File as CLEAN

END IF

ELSE

LogError(File)

END IF

END FOR

END

EVALUATION

Signature-based detection is highly effective for known malware because it compares files with signatures of previously identified threats. It can provide fast and reliable detection when the signature database is current.

Its main limitation is that it depends on known signatures. New, modified, or previously unseen malware may not match an existing signature and can therefore remain undetected until a suitable signature is created and added to the database. Regular signature updates are essential.

3. DDOS DETECTION USING TRAFFIC THRESHOLD ANALYSIS

Design pseudocode for a distributed denial-of-service (DDoS) detection system using traffic threshold analysis. Evaluate how the algorithm differentiates between legitimate high traffic and attack traffic.

PSEUDOCODE

ALGORITHM DDoSDetection

BEGIN

Set TrafficThreshold

Set TimeWindow

WHILE network monitoring is active DO

TrafficData = CollectTraffic(TimeWindow)

TrafficRate = CalculateTrafficRate(TrafficData)

SourceDistribution = AnalyzeSources(TrafficData)

ProtocolDistribution = AnalyzeProtocols(TrafficData)

IF TrafficRate > TrafficThreshold THEN

IF SourceDistribution is highly abnormal OR

ProtocolDistribution is highly abnormal THEN

Raise DDoSAlert

IdentifySuspiciousSources(TrafficData)

ApplyTrafficFiltering()

LogIncident(TrafficData)

ELSE

Mark traffic as POSSIBLY_LEGITIMATE

ContinueMonitoring()

END IF

ELSE

ContinueMonitoring()

END IF

END WHILE

END

EVALUATION

The algorithm detects possible DDoS attacks by comparing traffic volume with a predefined threshold. However, high traffic alone does not always indicate an attack because legitimate events can also produce large traffic volumes.

To differentiate legitimate high traffic from attack traffic, the system also examines factors such as source distribution and protocol patterns. Legitimate traffic generally has more expected patterns, while DDoS traffic may show abnormal or highly concentrated patterns. Thresholds should be configured carefully to reduce false positives and false negatives.

4. MACHINE LEARNING BASED NETWORK INTRUSION DETECTION SYSTEM

Design pseudocode for a network intrusion detection system using machine learning classification. Evaluate how feature selection and training data impact detection accuracy.

PSEUDOCODE

```
ALGORITHM MLIntrusionDetection
BEGIN
    TrainingData = LoadTrainingDataset()
    CleanData = Preprocess(TrainingData)
    Features = SelectRelevantFeatures(CleanData)
    Labels = ExtractLabels(CleanData)
    Model = TrainClassifier(Features, Labels)
    Evaluate(Model, TestDataset)
    WHILE network monitoring is active DO
        Traffic = CaptureNetworkTraffic()
        TrafficFeatures = ExtractFeatures(Traffic)
        SelectedFeatures = ApplySelectedFeatures(TrafficFeatures)
        Prediction = Model.Predict(SelectedFeatures)
        IF Prediction = INTRUSION THEN
            RaiseIntrusionAlert(Traffic)
            LogIncident(Traffic)
        ELSE
            Mark Traffic as NORMAL
        END IF
    END WHILE
END
```

EVALUATION

Feature selection directly affects detection accuracy. Relevant features can help the classifier identify meaningful patterns while removing unnecessary features can reduce complexity and improve efficiency. Poor feature selection may cause important attack characteristics to be missed. Training data is equally important. A dataset should contain representative examples of normal traffic and different types of attacks. If the training data is incomplete, biased, outdated, or imbalanced, the model may produce inaccurate classifications. Regular evaluation and suitable training data are therefore necessary for reliable intrusion detection.

5. SECURE NETWORK ACCESS CONTROL SYSTEM

Design pseudocode for a secure network access control system that authenticates users and enforces role-based access policies. Evaluate how access control mechanisms enhance overall network security.

PSEUDOCODE

ALGORITHM SecureNetworkAccessControl

BEGIN

Load UserDatabase

Load RolePermissions

User = ReceiveLoginRequest()

IF User exists in UserDatabase THEN

IF VerifyPassword(User) = TRUE THEN

IF MultiFactorAuthentication(User) = TRUE THEN

Role = GetUserRole(User)

Permissions = GetPermissions(Role)

CreateAuthenticatedSession(User, Role)

GRANT Permissions to User

WHILE Session is active DO

Request = ReceiveAccessRequest()

IF Request.Action is in Permissions THEN

ALLOW Request

LogAccess(User, Request)

ELSE

DENY Request

LogUnauthorizedAttempt(User, Request)

END IF

END WHILE

ELSE

DENY Access

LogFailedAuthentication(User)

END IF

ELSE

DENY Access

LogFailedAuthentication(User)

```
END IF
ELSE
DENY Access
LogUnknownUserAttempt(User)
END IF
END
```

EVALUATION

Access control improves network security by ensuring that only authenticated and authorized users can access network resources. Authentication verifies the identity of the user, while role-based access control determines which operations the user is permitted to perform.

Role-based access policies support the principle of least privilege because users receive only the permissions required for their roles. Logging access and unauthorized attempts also helps administrators monitor suspicious activity and investigate security incidents. Combining authentication with authorization therefore reduces the risk of unauthorized access and misuse of network resources.

OVERALL CONCLUSION

The five algorithms demonstrate different approaches to network and system security. Stateful firewall filtering provides session-aware packet protection, signature-based detection identifies known malware, traffic threshold analysis helps identify possible DDoS activity, machine learning classification supports intrusion detection, and role-based access control protects network resources through authentication and authorization. Each approach addresses a specific security requirement and becomes more effective when its limitations and operating conditions are properly considered.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient